Gestão de Peças, Qualidade e Armazenamento

PALOMA AI TSUCHINAGA

ALGORITMOS E LÓGICA DE PROGRAMAÇÃO

2026

UniFECAF

FACULDADE DE INTELIGÊNCIA ARTIFICIAL

Introdução

Imagine uma fábrica onde muitas peças passam por uma esteira todo dia. Cada uma delas precisa ter o peso certo, a cor correta e o tamanho dentro do esperado antes de seguir para a caixa. Fazer essa verificação manualmente, peça por peça, seria lento, cansativo e fácil de errar. É justamente aí que a automação faz diferença.

Este trabalho apresenta um programa desenvolvido em Python que simula um sistema de controle de qualidade. O programa inspeciona cada peça, aprova as que estão dentro dos critérios, reprova as que estão fora, organiza as aprovadas em caixas de dez unidades e gera um relatório ao final com tudo que foi processado.

Para construir essa solução foram usados conceitos básicos de lógica de programação: condições para tomar decisões, repetição para processar as peças uma a uma, funções para organizar o código e listas para guardar as informações. Simples na forma, mas com o mesmo raciocínio que está por trás dos sistemas automáticos usados na indústria hoje.

Importância da Automação

A automação industrial é um dos pilares do desenvolvimento econômico moderno, e sua importância se manifesta em diversas dimensões:

Aumento de produtividade: Máquinas e sistemas automatizados operam 24 horas por dia, 7 dias por semana, sem pausas ou fadiga. Isso permite volumes de produção muito maiores do que seria possível com mão de obra exclusivamente humana.

Redução de custos: Embora o investimento inicial seja elevado, a automação reduz custos operacionais a longo prazo — menos desperdício de materiais, menor consumo de energia por unidade produzida e redução de retrabalho por erros.

Qualidade e padronização: Processos automatizados executam tarefas com precisão milimétrica e consistência absoluta, eliminando a variabilidade humana e garantindo produtos dentro dos padrões de qualidade estabelecidos.

Segurança do trabalhador: Tarefas perigosas — como manuseio de produtos químicos, soldagem em altas temperaturas ou trabalho em ambientes insalubres — podem ser delegadas a robôs e sistemas autônomos, reduzindo acidentes de trabalho.

Competitividade global: Empresas que automatizam seus processos conseguem ofertar produtos a preços mais competitivos e com maior velocidade de entrega, o que é decisivo num mercado globalizado.

Rastreabilidade e dados: Sistemas automatizados geram grandes volumes de dados em tempo real, permitindo monitorar o processo produtivo, identificar falhas precocemente e tomar decisões baseadas em evidências.

Escalabilidade: A produção pode ser ampliada ou reduzida de forma mais ágil conforme a demanda do mercado, sem a complexidade de contratar ou demitir grandes equipes.

É claro que a automação também traz desafios, como a necessidade de requalificação profissional dos trabalhadores e o risco de desemprego em setores específicos. Por isso, ela deve ser acompanhada de políticas públicas e investimento em educação e capacitação.

Descrição Narrativa

1

Início
O programa começa cumprimentando o operador com uma mensagem de boas-vindas e já informa como encerrar o processo quando não houver mais peças. O ID da primeira peça começa em 1 e vai aumentando automaticamente a cada nova peça processada.

2

Entrada de uma nova peça
O operador informa se há uma nova peça na esteira. Se a resposta for "nao", o programa encerra o loop e segue direto para o relatório final. Se a resposta for "sim", o programa pede três informações: o peso em gramas, a cor e o comprimento em centímetros.

3

Verificação do peso
O programa verifica se o peso informado está entre 95g e 105g. Se estiver fora desse intervalo, o programa registra o motivo e marca a peça como reprovada. Se estiver dentro, segue para a próxima verificação.

4

Verificação da cor
O programa verifica se a cor informada é azul ou verde. Qualquer outra cor faz o programa registrar mais um motivo de reprovação. Se a cor estiver correta, segue para a próxima verificação.

5

Verificação do comprimento
O programa verifica se o comprimento está entre 10cm e 20cm. Se estiver fora, registra o motivo. Uma peça pode acumular reprovações das três verificações ao mesmo tempo, e o programa mostra cada motivo separado na tela.

6

Resultado da peça
Após as três verificações, o programa exibe o resultado. Se a peça foi reprovada em qualquer critério, ela é descartada e seu ID é guardado na lista de peças reprovadas. Se passou em todos os critérios, ela é aprovada e segue para a caixa.

7

Controle da caixa
Cada peça aprovada é adicionada à caixa atual e o contador sobe. O programa mostra quantas peças já estão na caixa no formato "1/10", "2/10" e assim por diante. Quando o contador chega em 10, o programa fecha a caixa, registra ela na lista de caixas fechadas, abre uma nova caixa vazia, zera o contador e avisa o operador.

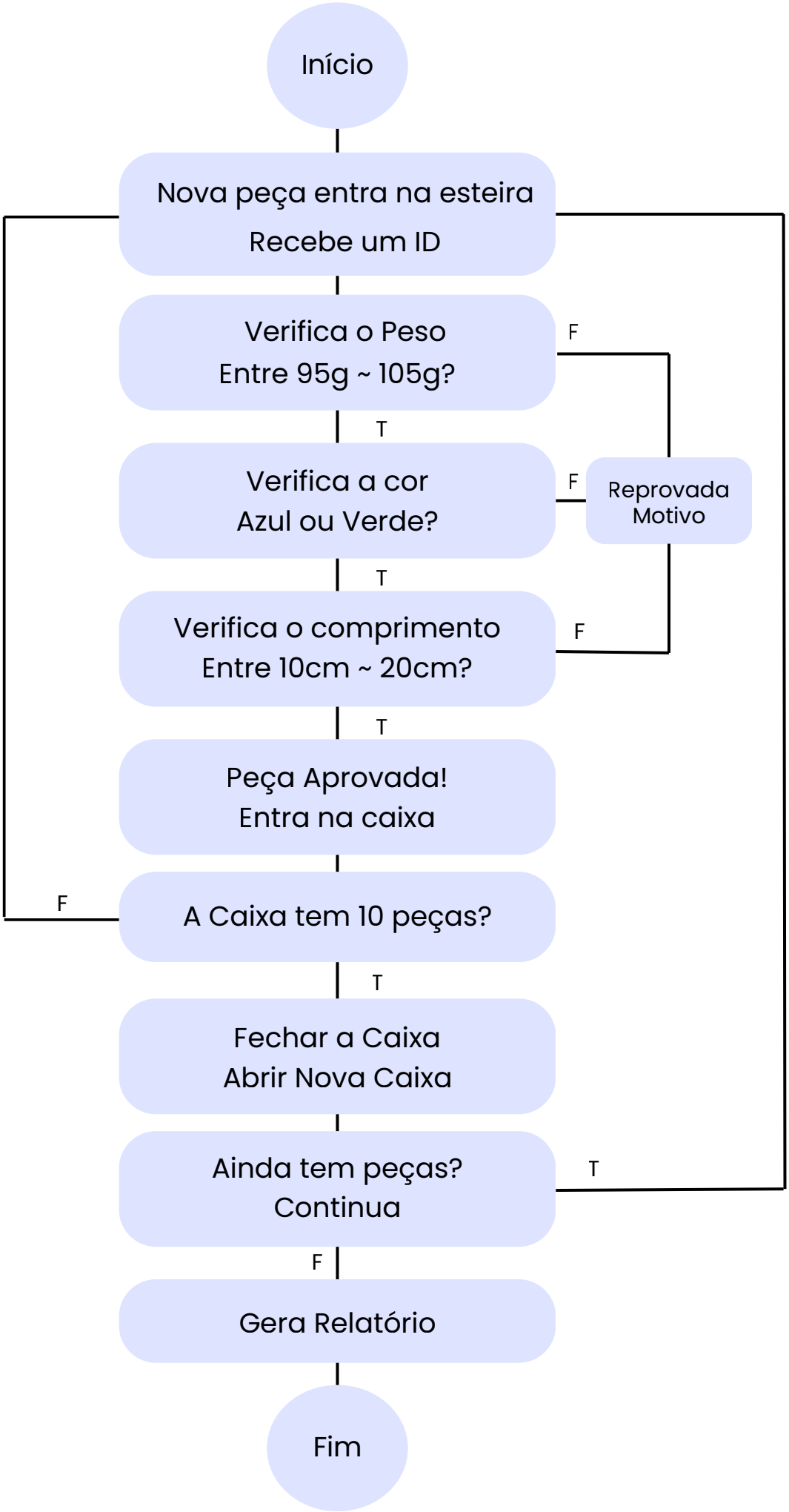
8

Retorno ao início do loop
Com a peça processada e a caixa atualizada, o ID aumenta em 1 e o programa volta ao início do loop, perguntando se há uma nova peça na esteira. Esse ciclo se repete até o operador digitar "nao".

9

Relatório final
Ao encerrar o loop, o programa calcula e exibe um resumo completo: total de peças processadas, total de aprovadas, total de reprovadas, total de caixas fechadas e, se houver peças na caixa no momento do encerramento, informa quantas ficaram na caixa incompleta. Se houve peças reprovadas, lista os IDs de todas elas.

Fluxograma



FLUXOGRAMA

Condições de Inspeção

Como o programa vai saber se uma peça está boa ou não?
A resposta foi usar condições, que são os if e and e or que aparecem no código.

Para o peso, por exemplo, escrevi assim:

```
# --- Está dentro do peso? ---
    if peso < 95 or peso > 105:
        reprovada = True
        print("  Motivo: Peso invalido:", peso, "g")
```

Para a cor eu usei:

```
# --- É azul ou verde? ---
    if cor != "azul" and cor != "verde":
        reprovada = True
        print("  Motivo: Cor invalida:", cor)
```

Aqui usei o **and** porque as duas condições precisam ser **verdadeiras ao mesmo tempo** — a cor não pode ser azul E também não pode ser verde. Se for qualquer outra coisa, reprova.

Para o comprimento eu usei:

```
# --- Tem entre 10 e 20cm de comprimento? ---
    if comprimento < 10 or comprimento > 20:
        reprovada = True
        print("  Motivo: Comprimento invalido:", comprimento, "cm")
```

Isso significa: se o comprimento for menor que 10 ou maior que 20, a peça está fora do padrão. Assim como no peso, usei o or porque qualquer um dos dois lados já é suficiente para reprovar.

Vale lembrar que as três verificações — peso, cor e comprimento — **acontecem sempre em sequência,** independente do resultado das anteriores. Isso significa que **uma peça pode ser reprovada por um motivo só, ou por dois ou três ao mesmo tempo**, e o programa mostra cada motivo separado na tela.

Funções

Quando percebi que o código estava ficando grande, pensei em como organizar melhor. Foi aí que separei tudo em duas funções.

FUNÇÃO VERIFICAR_PECA

A primeira, verificar_peca, é responsável por toda a inspeção. Ela recebe os dados da peça, verifica cada critério e devolve uma resposta: aprovada ou reprovada. Colocar isso numa função separada foi importante porque esse bloco é chamado várias vezes durante o programa, uma vez para cada peça. Se eu tivesse escrito esse código solto, teria que repetir tudo a cada peça processada.

ANTES - SEM FUNÇÃO

```
# FUNÇÃO DA INSPEÇÃO DA PEÇA
# ____________________________________________

peso = int(input("Informe o PESO da peca (g): "))
cor = input("Informe a COR da peca (azul/verde): ")
comprimento = int(input("Informe o COMPRIMENTO da peca (cm): "))

# --- Verificar Peça 1 --
if peso < 95 or peso > 105:
    print("Motivo: Peso invalido:", peso, "g")
if cor != "azul" and cor != "verde":
    print("Motivo: Cor invalida:", cor)
if comprimento < 10 or comprimento > 20:
    print("Motivo: Comprimento invalido:", comprimento, "cm")

# --- Verificar Peça 2 --
peso = int(input("Informe o PESO da peca (g): "))
cor = input("Informe a COR da peca (azul/verde): ")
comprimento = int(input("Informe o COMPRIMENTO da peca (cm): "))

if peso < 95 or peso > 105:
    print("Motivo: Peso invalido:", peso, "g")
if cor != "azul" and cor != "verde":
    print("Motivo: Cor invalida:", cor)
if comprimento < 10 or comprimento > 20:
    print("Motivo: Comprimento invalido:", comprimento, "cm")
```

E continuava, peça 3 em diante...

```python
# FUNÇÃO DA INSPEÇÃO DA PEÇA
# ____________________________________________

def verificar_peca(id, peso, cor, comprimento):
    reprovada = False

# --- Está dentro do peso? ---
    if peso < 95 or peso > 105:
        reprovada = True
        print("  Motivo: Peso invalido:", peso, "g")

# --- É azul ou verde? ---
    if cor != "azul" and cor != "verde":
        reprovada = True
        print("  Motivo: Cor invalida:", cor)

# --- Tem entre 10 e 20cm de comprimento? ---
    if comprimento < 10 or comprimento > 20:
        reprovada = True
        print("  Motivo: Comprimento invalido:", comprimento, "cm")


# --- Aprovada ou Reprovada? ---
    if reprovada:
        pecas_reprovadas.append(id)
        print("  Resultado: REPROVADA")
    else:
        print("  Resultado: APROVADA")

    return not reprovada
```

Perceba que o mesmo bloco de verificação se repete inteiro para cada peça. Se o programa precisasse processar 100 peças, esse bloco apareceria 100 vezes.

FUNÇÃO GERAR_RELATORIO

A segunda, gerar_relatorio, cuida apenas de mostrar o resumo final. Ela usa as listas e variáveis que foram sendo preenchidas ao longo do programa para montar as informações. Separar o relatório numa função própria deixou o programa principal mais limpo e fácil de acompanhar.

ANTES - SEM FUNÇÃO

```
# --- FUNÇÃO RELATÓRIO FINAL ---
# ____________________________________________

total_caixas = len(caixas_fechadas)
total_aprovadas = total_caixas * 10 + contador_caixa
total_reprovadas = len(pecas_reprovadas)
total_processadas = total_aprovadas + total_reprovadas

    print("")
    print("      RELATORIO FINAL DE PRODUCAO")
print("Total de pecas processadas :", total_processadas)
print("Total de pecas aprovadas   :", total_aprovadas)
print("Total de pecas reprovadas  :", total_reprovadas)
print("Total de caixas fechadas   :", total_caixas)

# --- Se sobraram peças ---
if contador_caixa > 0:
    print("Pecas na caixa atual       :", contador_caixa, "(incompleta)")

# --- Peças que foram reprovadas ---
if total_reprovadas > 0:
    print("")
    print("IDs das pecas reprovadas:")
    for id in pecas_reprovadas:
        print("  - ID", id)

    print("")
    print("Programa finalizado.")
```

Nesse caso funciona, porque o relatório aparece apenas uma vez no programa. O problema é que esse bloco grande fica misturado com o restante do código, deixando o programa principal mais difícil de ler e acompanhar.

DEPOIS - COM FUNÇÃO

```python
# --- FUNÇÃO RELATÓRIO FINAL ---
# ____________________________________________

def gerar_relatorio():
    total_caixas = len(caixas_fechadas)
    total_aprovadas = total_caixas * 10 + contador_caixa
    total_reprovadas = len(pecas_reprovadas)
    total_processadas = total_aprovadas + total_reprovadas

# --- Total de Peças --

    print("")
    print("      RELATORIO FINAL DE PRODUCAO")
    print("Total de pecas processadas :", total_processadas)
    print("Total de pecas aprovadas   :", total_aprovadas)
    print("Total de pecas reprovadas  :", total_reprovadas)
    print("Total de caixas fechadas   :", total_caixas)

# --- Se sobraram peças ---
    if contador_caixa > 0:
        print("Pecas na caixa atual       :", contador_caixa, "(incompleta)")

# --- Peças que foram reprovadas ---
    if total_reprovadas > 0:
        print("")
        print("IDs das pecas reprovadas:")
        for id in pecas_reprovadas:
            print("  - ID", id)
    print("")
    print("Programa finalizado.")
```

A diferença aqui é mais sobre organização do que sobre repetição.

Todo aquele bloco grande de código fica guardado dentro da função, e no programa principal aparece apenas uma linha: gerar_relatorio().

Quem lê o programa principal entende na hora o que acontece ali, sem precisar ler linha por linha do relatório.

Explicando a Repetição

Repetição — fazendo o programa processar várias peças

O programa precisava processar não uma, mas várias peças em sequência. Para isso usei o **while True**, que cria um loop que fica rodando sem parar até que algo mande parar.

O que manda parar é o operador digitando nao quando não há mais peças. Nesse momento o **break *encerra o loop*** e o programa **segue para o relatório**. Dentro do loop, a cada volta, o programa pede os dados de uma nova peça, verifica, decide o que fazer e já **incrementa o ID para a próxima**.

Explicando o uso de Arrays

Arrays ou Listas — guardando as informações ao longo do processo

Para o relatório final funcionar, o programa precisava lembrar de tudo que aconteceu. Para isso usei três arrays: um para guardar as peças dentro da caixa atual, um para guardar as caixas já fechadas e um para guardar os IDs das peças reprovadas.

A lógica da caixa foi a parte que exigiu mais atenção.

Criei uma variável contador_caixa que vai somando a cada peça aprovada. Quando ela chega em 10, o programa fecha a caixa, salva ela na lista de caixas fechadas, abre uma nova caixa vazia e zera o contador. Assim o ciclo recomeça automaticamente.

```
caixa_atual = []        # Quantidade de peças na caixa
caixas_fechadas = []    # Quantidade de caixas fechadas
pecas_reprovadas = []   # Lista de peças reprovadas
id_peca = 1             # Contador de ID para cada peca
contador_caixa = 0      # Conta quantas peças estão na caixa atual
```

Aplicação do Programa em um Cenário Real

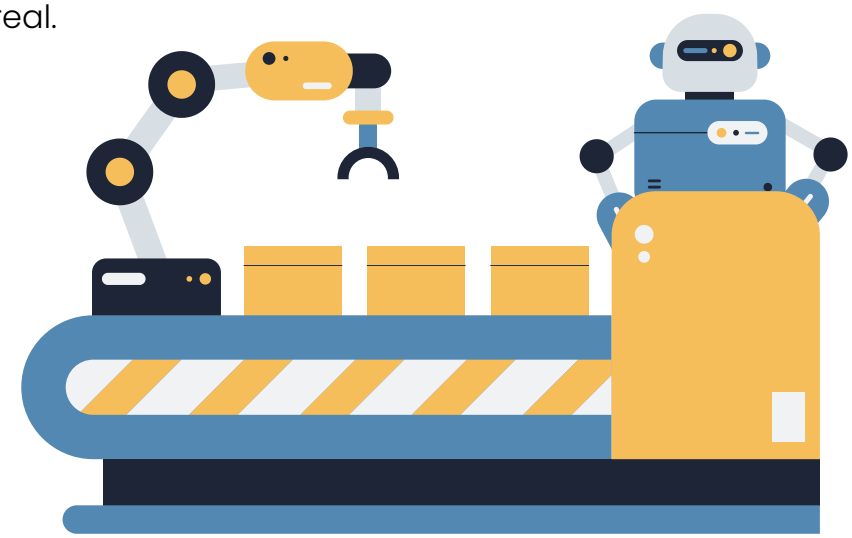
O programa desenvolvido neste projeto vai muito além de um exercício de aula. A lógica aplicada aqui — verificar, decidir, aprovar, descartar e registrar — é exatamente a mesma que opera em linhas de produção industriais ao redor do mundo.

Na prática, o que foi feito com **input** e **print** seria substituído por equipamentos reais. **Sensores de peso**, **cor** e c**omprimento** enviariam os dados diretamente para o programa, sem a necessidade de um operador digitando as informações.

As decisões que o código toma com os **if** seriam traduzidas em comandos elétricos para os equipamentos responsáveis por movimentar, aprovar ou descartar cada peça. Como motores, atuadores pneumáticos, hidráulicos e esteiras. Toda a parte mecânica já está resolvida pelo hardware. O que esse programa fornece é a lógica que diz quando e como cada ação deve acontecer.

E é justamente aí que entra o conceito aprendido em aula: uma máquina inteligente é aquela capaz de tomar decisões de forma autônoma. Com a lógica correta e uma boa gestão de riscos, esse programa poderia ir além das regras fixas. **Ele poderia aprender com o histórico de produção, identificar padrões de defeitos, ajustar os critérios automaticamente e operar sem intervenção humana**. Isso é o que acontece nas indústrias mais avançadas hoje, onde sistemas baseados nessa mesma lógica produzem milhares de peças por hora, com precisão constante e praticamente sem parar.

O que foi construído aqui com ferramentas simples e conceitos básicos de programação representa, em essência, o funcionamento de um sistema industrial real.



Benefícios Percebidos

Aprender a usar condições, repetição, funções e listas separadamente é uma coisa. O grande benefício foi perceber que, juntando tudo isso, já é possível construir algo que resolve um problema real.

Com apenas o que foi visto em aula, foi possível criar um programa que toma decisões sozinho, processa várias situações em sequência, organiza informações e gera um resultado no final. Isso mostra que a base aprendida não é só teoria, ela já é suficiente para automatizar tarefas que hoje muitas empresas ainda fazem manualmente.

Outro benefício foi entender que a lógica aprendida aqui é a mesma usada em sistemas profissionais. A diferença é o tamanho e os equipamentos conectados, não o raciocínio por trás.

Desafios Enfrentados

O maior desafio foram as funções. Entender como criá-las, o que colocar dentro delas e como fazê-las conversar com o restante do programa não foi simples no início. Foi difícil perceber onde uma função começa e termina, o que ela precisa receber e o que ela precisa devolver para que o programa todo funcione corretamente.

Com o tempo e a prática foi ficando mais claro que a função é apenas um bloco com uma tarefa específica, e que o programa principal só precisa chamá-la quando necessário. Mas chegar nesse entendimento foi o ponto que mais exigiu atenção durante o desenvolvimento.

2026